

EECS 16B Final Lab Report

Aryaman Darda (3035703514): aryaman_darda@berkeley.edu,

John Glen Siy (3036909952): johnglen_siy@berkeley.edu,

Group ID: 10

1 System ID

How did you choose the region to collect finer data on `data_fine.text`? Why is it important to choose such a region to run least-squares regression on?

The finer data collects the car's velocity values over a smaller PWM range, a range of PWM values that the car will operate in. For our car, we chose a region of 50 to 100 PWM, which contains a midpoint PWM value that the car uses when it is moving straight. This region was adjusted to be wide enough so that the range contained multiple overlapping velocities for both wheels to operate on. It is important to select a region that has overlapping velocities since it provides us with a margin of error on both sides of the operating velocity achievable by both wheels. We imagine that since we are approximating the car's movement, the model won't exactly move straight at our midpoint and the overlapping region is used to compensate for this.

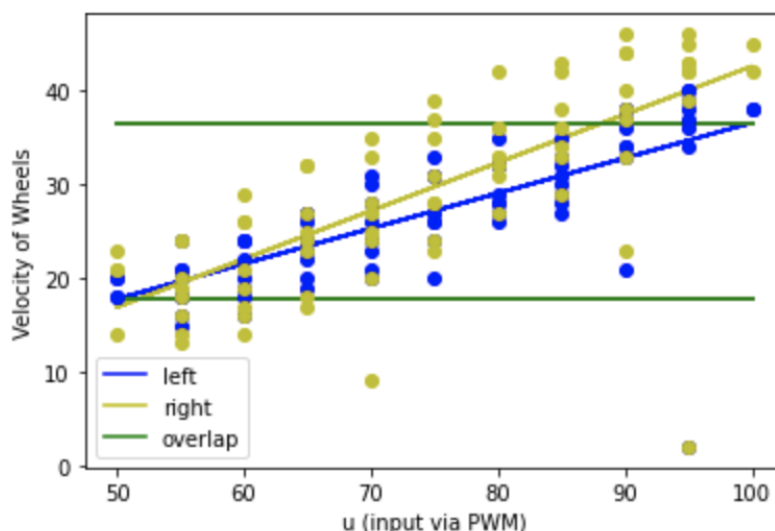


Figure 1. Green lines denote the overlapping region of velocities that are reachable by both wheels from a PWM 50-100.

What do θ and β represent physically, not mathematically?

The θ parameter represents how much our velocity changes as the input PWM changes. Our model assumes that velocity is directly proportional to PWM and that it increases linearly as

PWM increases. The β parameter represents the constant negative offset that our velocity has due to the static friction that prevents the car from moving from rest at particularly low PWM values. Our car doesn't start moving at PWM values close to 0, and the model compensates for this by adding a negative velocity.

Why do we have separate θ and β values for the left and right wheels?

Our model isn't an exact model of how the left and right wheels move, it's only a rough representation of it. Due to varying physical phenomena independent to each wheel, such as each wheel's friction coefficients, the weight distribution on each car's side, or the internal properties of each motor, the θ and β values for each wheel don't match.

Why do we set v^* to the midpoint of our overlapping wheel velocity range, instead of closer to the boundaries? What would happen if we operated our car at a velocity outside of the overlapping velocity range?

V^* is the midpoint of the overlapping wheel velocity range since it gives us the most margin of error for each motor to operate in. The velocities of each wheel don't match the inputs of the motors according to the model every time, so the overlapping range is responsible for giving the motors multiple PWM values to work with around the operating point. If we chose an operating point outside our overlapping velocity range, it would be impossible for the car to drive straight, since the velocity will only be reachable by one wheel.

2 Controls Part 1

What are the open-loop model equations for our PWM input, $u[i]$?

$$v_L[i] = \theta_L u_L[i] - \beta_L$$

$$v_R[i] = \theta_R u_R[i] - \beta_R$$

These equations linearly relate PWM input to the wheel's velocities and are derived using linear least squares. We established how θ represents how much our velocity changes according to an increase in PWM input, hence θ is the mathematical slope in our model. β is mathematically the y-intercept since it represents a velocity offset for all PWM inputs.

What is the purpose of the jolts? Why might the left and right jolts be different?

The car starts from rest so it must first overcome static friction to start moving. However, the operating range that we've selected for our car doesn't supply a high enough PWM signal to overcome this friction, so we supply a PWM signal larger than our range for a short amount of time to get the car moving. The left and right jolts are different due to varying physical phenomena and variance in how our wheels individually respond to a PWM signal. One motor may require a larger PWM signal to get its wheel moving compared to the other motor.

Why does open-loop control fail? Why do we need to implement closed-loop control in order to have the car travel straight?

Our open-loop model makes assumptions about the movement of our car, particularly that the wheels' velocities correlate precisely linearly with our PWM inputs. However, the real-life movement of our car never actually follows this model exactly, considering that it encounters disturbances and noise from its environment. Closed-loop control is a modified version of our open-loop model that modified the input values to each wheel based on an error value - which we calculate as the differences in velocities of each wheel.

What are the closed-loop model equations for our PWM input, $u[i]$? Explain the purpose of each term.

$$u_L[i] = \frac{1}{\theta_L} (v^* - f_L \delta[i] + \beta_L)$$
$$u_R[i] = \frac{1}{\theta_R} (v^* + f_R \delta[i] + \beta_R)$$

u_L and u_R represent the same PWM inputs which unlike our open-loop model are now a function that varies with time-step. v^* is our target velocity and the midpoint of our overlapping range which we determined in the previous lab. $\delta[i]$ is the control variable identifying our car's behavior. We calculate the control variable as the difference in the distance traveled between the left and right wheels over all timesteps leading up to i . f_L and f_R are coefficients to be selected such that the control variable stays constant and no wheel travels farther than the other. β and θ are still the open-loop parameters we determined in linear-least squares. The key to our closed-loop model is that we have strategically added an extra variable term to our target velocity that essentially indicates to the car that it isn't moving straight, and this term depends on the control variable δ .

When testing out different f-values in practice, how do you know if the system eigenvalue has gone from positive to negative based on the car's behavior?

A positive system eigenvalue ensures that the state variable is positive for all time-steps. Assuming that the system is stable and that δ is positive, the car will continue decreasing the left wheel's velocity and increasing the right wheel's velocity until δ is driven to zero. A negative system eigenvalue on the other hand causes the state variable to switch signs every time step. With a stable system, δ will still converge to zero but the car will oscillate between turning left and right until it moves straight.

What effect does setting both f-values to 0 have on the car's control scheme? How is this different from non-zero f-values? Why are non-zero f-values necessary?

Setting both f-values to zero essentially reduces the car's control scheme back to the open-loop equations. If we plug in zero for each f-value in our closed-loop equations, we also set the term $f_{L,R}\delta[i]$ to 0, basically removing any adjustment to each wheel's velocity.

Why can't we use negative f-values for both wheels? If we wanted to use negative f-values for both wheels, how should we change our closed-loop model equations such that our car goes straight and corrects any errors in its trajectory?

Using negative f-values results in the magnitude of the system eigenvalue to be greater than 1. This guarantees the unstable condition for our car, where the magnitude of δ will exponentially increase instead of being driven down to zero. If for some reason we wanted to use negative f-values, we would change δ to be the right wheel's distance traveled minus the left wheel's distance traveled. This changes our system eigenvalue to $(-1 - f_L - f_R)$, allowing us to use negative f-values while maintaining a stable or marginally stable condition.

What does a zero delta ss value tell you about your car's trajectory? What about a non-zero delta ss value? What kind of error is it supposed to correct when we add it to our control scheme?

A zero δ_{ss} value indicates that the car is moving perfectly straight, as no wheel traveled farther than the other. A non-zero δ_{ss} value indicates that the car eventually went straight but not in the direction it was initially pointed at. One wheel traveled a farther distance than the other before the car corrected the wheels' velocities and this distance was kept constant. More often than not, the car will have a non-zero δ_{ss} value so we add this to our general δ term so that the car travels in the same direction it is pointed in.

3 Controls Part 2

How did you change the closed-loop model equations to allow the car to turn? Write the equations below and explain how they change for turning left, turning right, and going straight.

$$\begin{aligned}u_L[i] &= \frac{1}{\theta_L} (v^* - f_L \delta[i] + \beta_L) \\u_R[i] &= \frac{1}{\theta_R} (v^* + f_R \delta[i] + \beta_R) \\\delta[i] &= \delta[i] + \delta_{ref}\end{aligned}$$

These were the original closed-loop model equations but we added an extra term δ_{ref} to our δ at each time step to trick the car into thinking it is turning in a particular direction and it will try to correct this by turning in the direction that is desired.

When going straight, the closed-loop model equations do not change and δ_{ref} is set to 0 in both equations.

When turning right, we want $\delta[i]$ to be negative so we add a negative δ_{ref} and the closed loop equations become

$$\begin{aligned}u_L[i] &= \frac{1}{\theta_L} (v^* + f_L \delta[i] + \beta_L) \\u_R[i] &= \frac{1}{\theta_R} (v^* - f_R \delta[i] + \beta_R)\end{aligned}$$

Which makes the left wheel move at a quicker rate than the right wheel, hence turning right.

When turning left, we want $\delta[i]$ to be positive so we add a positive δ_{ref} and the closed loop equations remain the same but the right wheel now moves at a faster rate than it was previously at and the left wheel moves at a slower rate than what it was previously at. The magnitude of $\delta[i]$ increases and results in the car turning left to correct the “trick” right turn.

Why do we divide v^* by $m = 5$ for the turning expressions?

This is because the data collection collected data in 0.5 second intervals while the controller updated the trajectory every 0.1 seconds. There are discrepancies in the sampling periods which need to be fixed in order to get our car traveling at the same ground velocity and maintain a coherent definition of a timestep which is driven by these sampling periods. To make sure our data collector records the same velocity at 0.5s as our controller would in 0.1s, we need to divide the v^* by 5 to maintain coherence in the velocity value between the two different sampling periods.

How is using STRAIGHT CORRECTION different from delta ss in Controls Part 1?

The need to implement STRAIGHT_CORRECTION for the car comes from mechanical issues like axle wobble and tire size differences that could exist in the car. The encoder measurements might be the same but these mechanical shortcomings could still make the car turn gradually. To correct for this, STRAIGHT_CORRECTION is used to add a bias to the direction of motion of the car over a long period of time in a direction opposite to the one which it is turning in right now. This is different from delta ss because delta ss corrects for the steady state error that arises due to a mismatch between the model and the real world workings of the car. Delta ss corrects for when the model predicts the car as driving straight but in reality, the encoder values will be slightly mismatched and the car will not initially drive straight in the intended direction. STRAIGHT_CORRECTION corrects for the slight turn due to the mechanics of the car even when both encoder values are the same

5 Integration/Final Demo

Briefly discuss what you learned throughout the S1XT33N car project and in the labs. What was your favorite part? Least favorite part? Please answer this question individually.

Aryaman: The main thing I learned was how to put together the many pieces of circuitry and modeling that we learned in lecture into building an actual machine. The coherence and connection of the many topics was one thing that really stood out to me. I learned how to correctly and systematically debug circuits and the classification aspect of the labs was also really interesting and allowed me to get more hands on with learning models. My favourite part was definitely doing the PCA via SVD as classification as a whole is a field of great interest to me. My least favourite part was definitely implementing the mic board and related circuits as I find circuitry rather tedious especially when you're working with sound and frequencies.

John: Throughout the S1XT33N car project I learned how to transform and apply pretty much every abstract concept learned in class to a real hands-on project that was satisfying to complete in the end. We learned how to build an RLC circuit and compute its impedances in order to form a functional filtering micboard. After working through the physical circuitry we reviewed techniques for processing and manipulating electrical signals. We used linear least-squares to generate a model for driving straight and turning and a PCA classification algorithm to classify - satisfyingly - our voice commands for the robot. I agree with Aryman that the last few labs

involving PCA and SVD were my favorite part of lab. Projects involving classifying and manipulating data like these tend to pique my interest, and the coding and linear algebra techniques used had me wondering if I could apply them to other projects I hope to do in my free time. My least favorite part was by far having to debug the mic board multiple times in lab. I think it's more satisfying to debug something through code where you can at least insert statements in places to ensure that the code is working. It also felt tedious and frustrating to have to take apart parts of the circuit at times and I wasn't able to appreciate some of the abstractness involved. All in all, I actually felt that lab was the most fun part of the course and there was no particular lab that I didn't enjoy.

What was the most difficult bug you encountered this semester? How did you resolve the bug? What did you learn from the debugging experience?

The most difficult bug we encountered was definitely getting our classification code to work. There was no lab held so there was no TA support. We were struggling with implementing the code to cluster the data around centroids and also choosing a basis using principal components. We realized that we needed to read through the lab note carefully and re-visit lectures to solidify the notion of SVD. This took some time but we finally fully understood the concept and were able to figure the code out. I guess the thing we learned from this process is that we should always be conceptually prepared before going for lab because a strong foundation on the concepts means that one better understands how and why they are applied in a practical situation.

6 Feedback

Please provide any feedback you have about the 16B lab or anything we can do to better support you.

The labs were running pretty smoothly before the strike so under normal circumstances, there is no real feedback on our end.

7 Collaborators and Sources

Please detail each group member's contributions to the lab report. Also, cite any sources you used that were not provided with the course materials.

Aryaman: Controls part 2, Integration debugging question

John: System ID, Controls part 1

All sources were from the course materials!